LEDGER BANCAR

Badea Cornel-Florin

PROJECT OVERVIEW

- A high-performance banking ledger built to handle high-concurrency environments safely.
- Backend: Python (FastAPI) for rapid asynchronous processing.
- Database: PostgreSQL for ACID compliance and hard data storage.
- Caching: Redis for lightning-fast idempotency checks.
- Frontend: Bootstrap 5 UI, fully decoupled via CORS.

CORE OBJECTIVES

1. Zero Data Loss: Ensure every cent is accounted for under heavy load.

2. Concurrency Control: Prevent Race Conditions when multiple requests hit the same account simultaneously.

1. Spam Prevention: Implement Idempotency to stop accidental or malicious double-charging.

2. Real-time Visibility: Provide a live interface that reflects the exact state of the database.

HOW TO RUN THE PROJECT

1. Start Services: Ensure PostgreSQL and Redis servers are running locally.

2. Environment: Activate the Python virtual environment (source venv/Scripts/activate).

3. Launch Backend: Execute uvicorn main:app --reload to start the API on port 8000.

4. Launch Frontend: Open the index.html file directly in any web browser.

5. Execute Tests: Open a separate terminal and run k6 run [spam(numeber)_test].js.

LOAD TESTING STRATEGY

- To prove the architecture's resilience, the system is subjected to stress tests using Grafana K6.
- Virtual Users (VUs): Simulating 100+ concurrent connections mimicking real-world traffic.
- Volume: Processing thousands of transactions in a matter of seconds.
- Environment: Fully local execution (127.0.0.1) utilizing raw CPU power.

TEST VULNERABILITIES

1. Race Conditions: If two transfers happen at the exact same millisecond, standard databases overwrite each other, creating "phantom money".

2. Idempotency Failures: Network delays can cause a user to click "Send" twice. Without filters, the system processes the same payment multiple times.

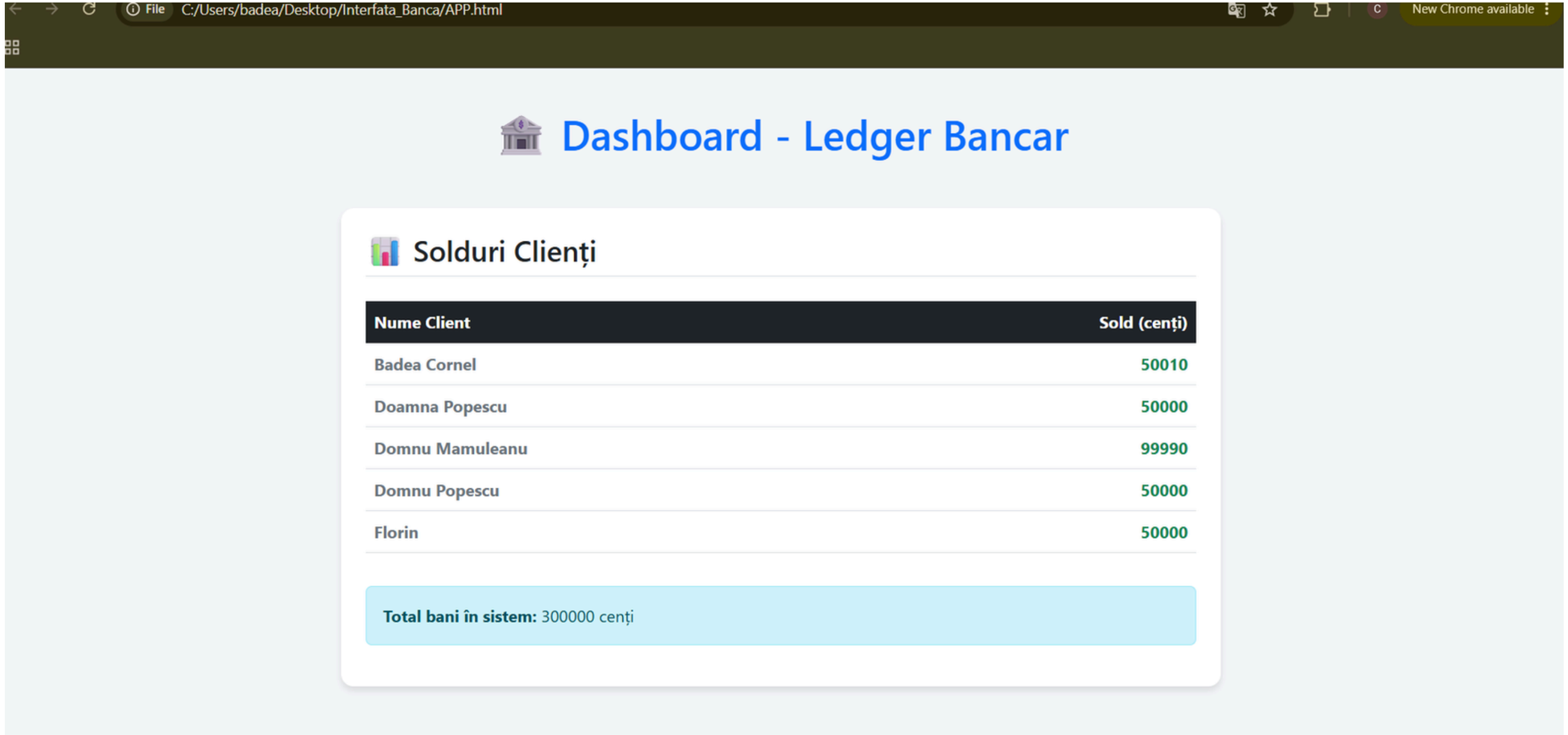
DUAL VERIFICATION SYSTEM

We validate the success and failures of transactions through two distinct layers:

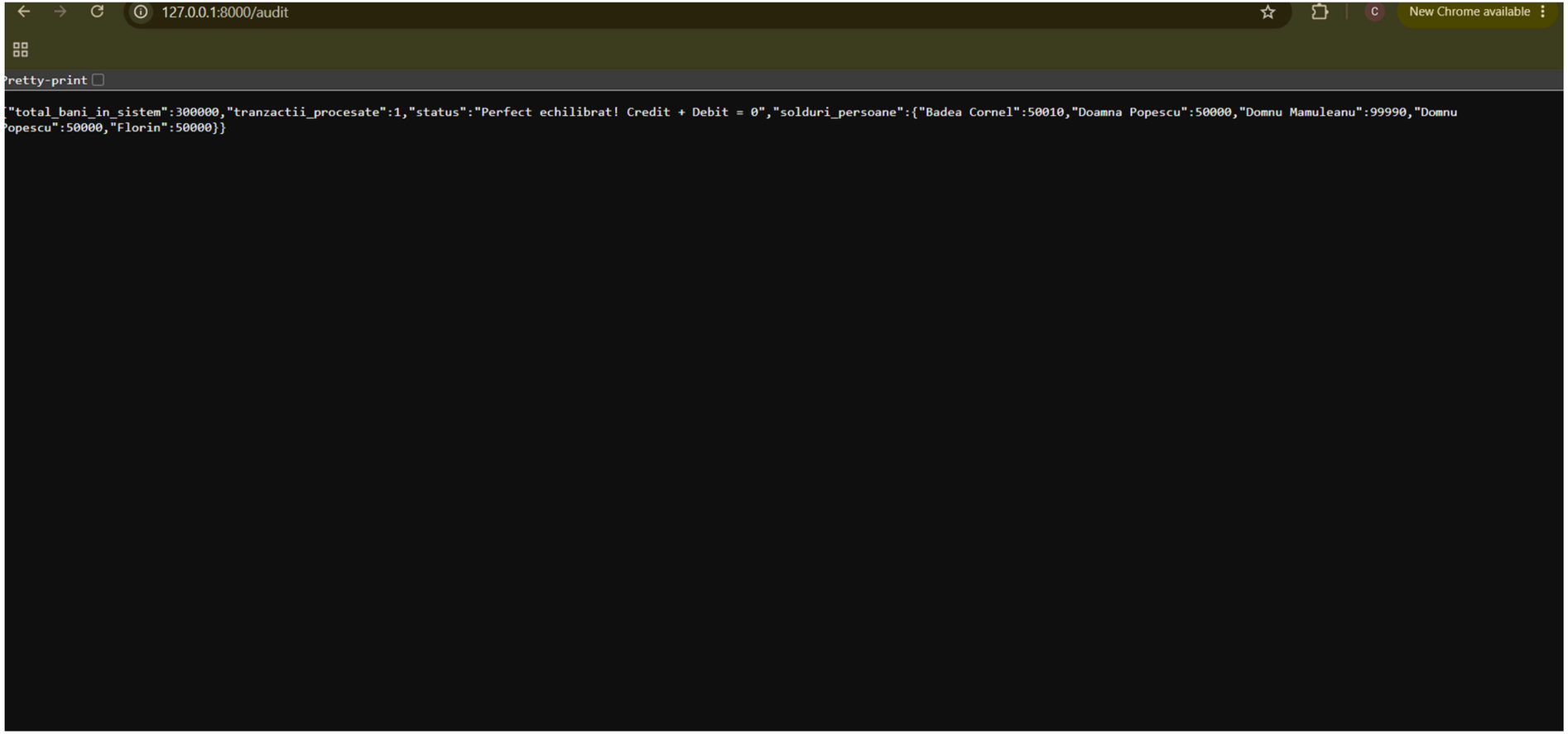
1. Visual Interface (Dashboard): The decoupled Frontend continuously fetches real-time balances. We can visually confirm that no account drops below zero.

2. Localhost Audit: Querying the PostgreSQL database directly and checking the server logs ensures the mathematical integrity is perfectly maintained.

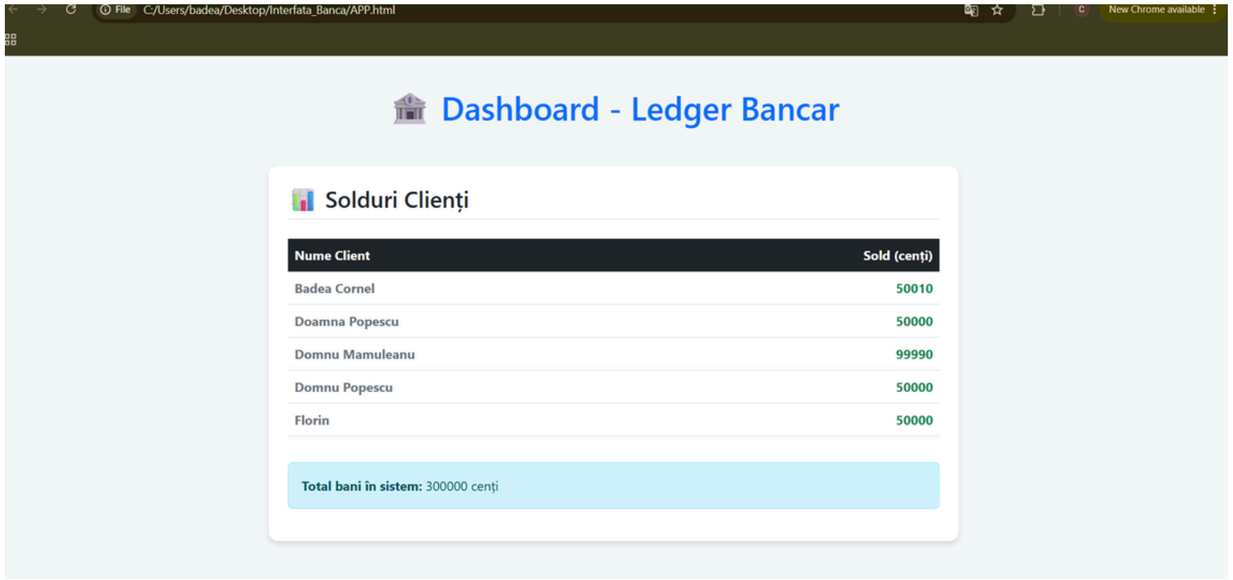
1. Visual Interface (Dashboard):

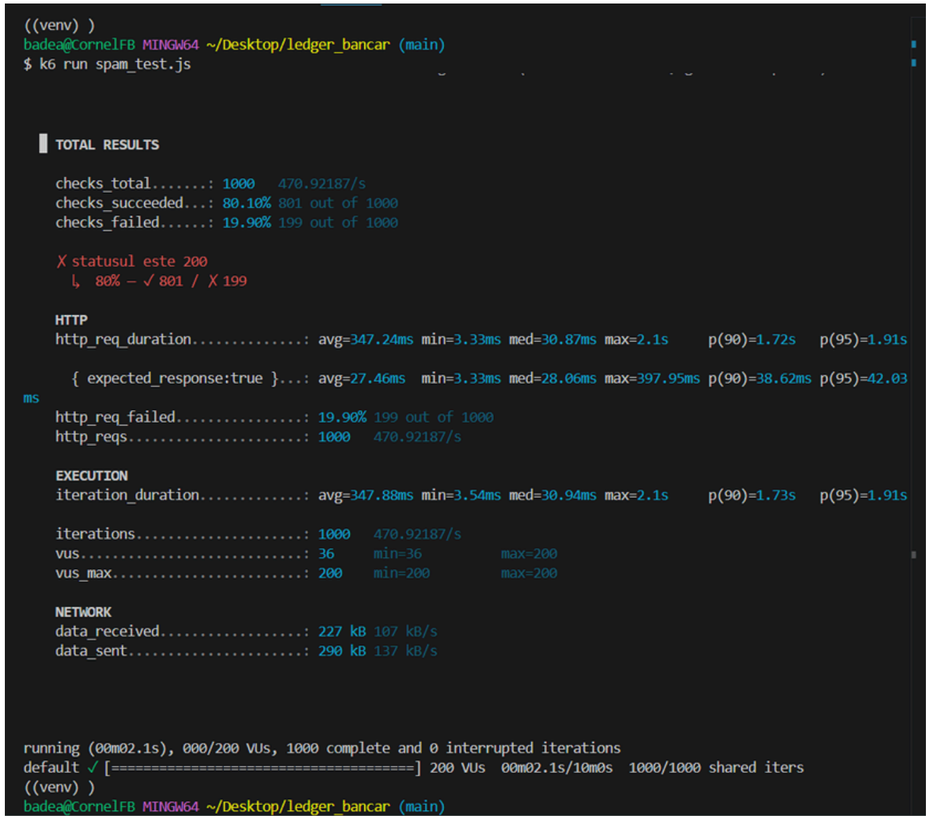


2. Localhost Audit:

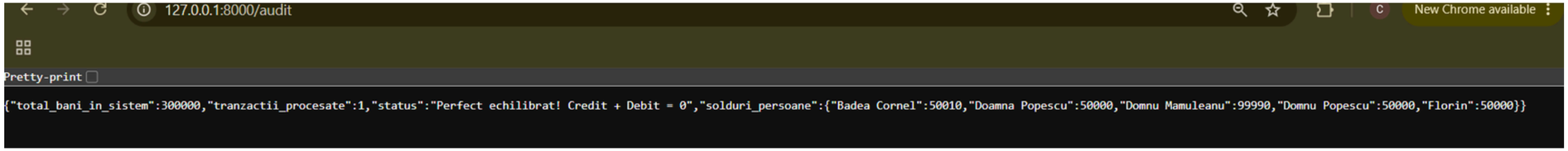


TEST 1: SPAM & IDEMPOTENCY





In this scenario, I simulated a Spam attack with 1,000 transactions sent by 200 concurrent users. Even though K6 shows 80% valid responses from the server and 20% dropped connections due to localhost traffic limits, the true metric of success is in the database audit. Out of 1,000 attacks, the Redis shield blocked 999, allowing only a single valid transaction to pass. The funds are 100% secure.



TEST 2: CONCURRENCY & REJECTION LOGIC

Dashboard - Ledger Bancar

Solduri Clienți

Nume Client	Sold (cenți)
Badea Cornel	50315
Doamna Popescu	48399
Domnu Mamuleanu	101960
Domnu Popescu	50284
Florin	49042

Total bani în sistem: 300000 cenți

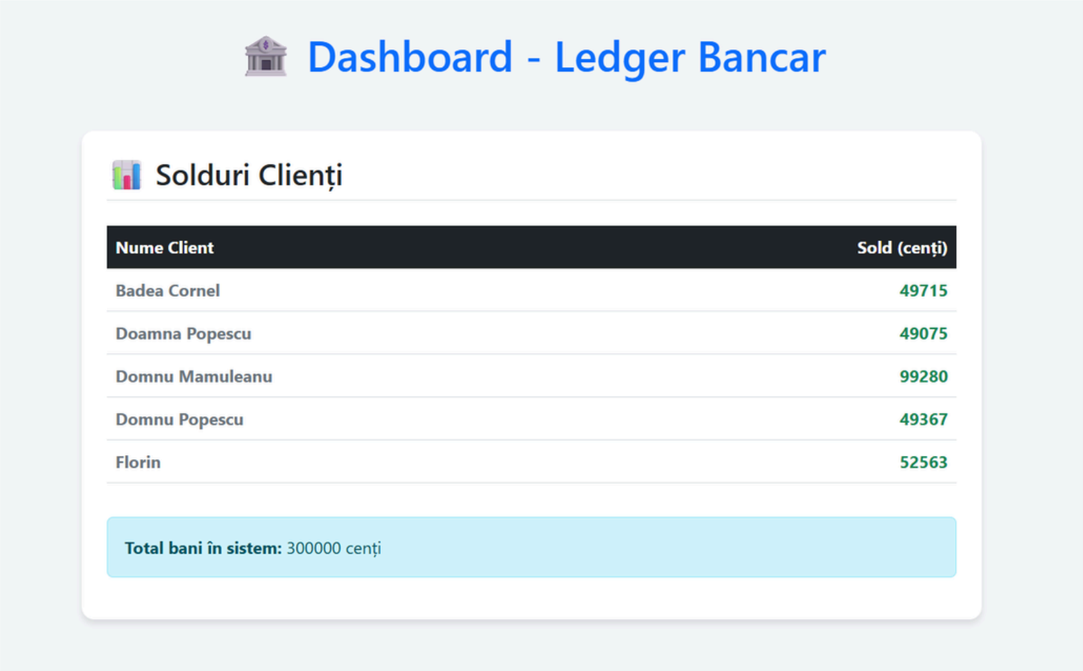
```
PROBLEMS 1  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

((venv) )
badea@CornelFB MINGW64 ~/Desktop/ledger_bancar (main)
$ k6 run spam2_test.js

          * default: 100000 iterations shared among 100 VUs (maxDuration: 10m0s, gracefulStop: 30s)


  TOTAL RESULTS

    checks_total.......: 22493  28.035258/s
    checks_succeeded...: 96.74% 21761 out of 22493
    checks_failed......: 3.25%  732 out of 22493

    ✗ statusul este 200
      ↳  96% — ✓ 21761 / ✗ 732

    HTTP
    http_req_duration..............: avg=2.19s min=10.98ms med=496.5ms  max=35.68s p(90)=5.81s p(95)=8.38s
      { expected_response:true }...: avg=2.06s min=10.98ms med=471.38ms max=35.68s p(90)=5.44s p(95)=8.3s
    http_req_failed................: 3.25%  732 out of 22493
    http_reqs......................: 22493  28.035258/s

    EXECUTION
    dropped_iterations.............: 77407  96.480027/s
    iteration_duration.............: avg=2.19s min=11.06ms med=496.7ms  max=35.68s p(90)=5.81s p(95)=8.39s
    iterations.....................: 22493  28.035258/s
    vus............................: 0      min=0           max=100
    vus_max........................: 100    min=100         max=100

    NETWORK
    data_received..................: 5.4 MB 6.7 kB/s
    data_sent......................: 7.1 MB 8.9 kB/s



running (13m22.3s), 000/100 VUs, 22493 complete and 100 interrupted iterations
default ✗ [======>----------------------------] 100 VUs  13m21.7s/10m0s  022493/100000 shared iters
((venv) )
badea@CornelFB MINGW64 ~/Desktop/ledger_bancar (main)
$
```

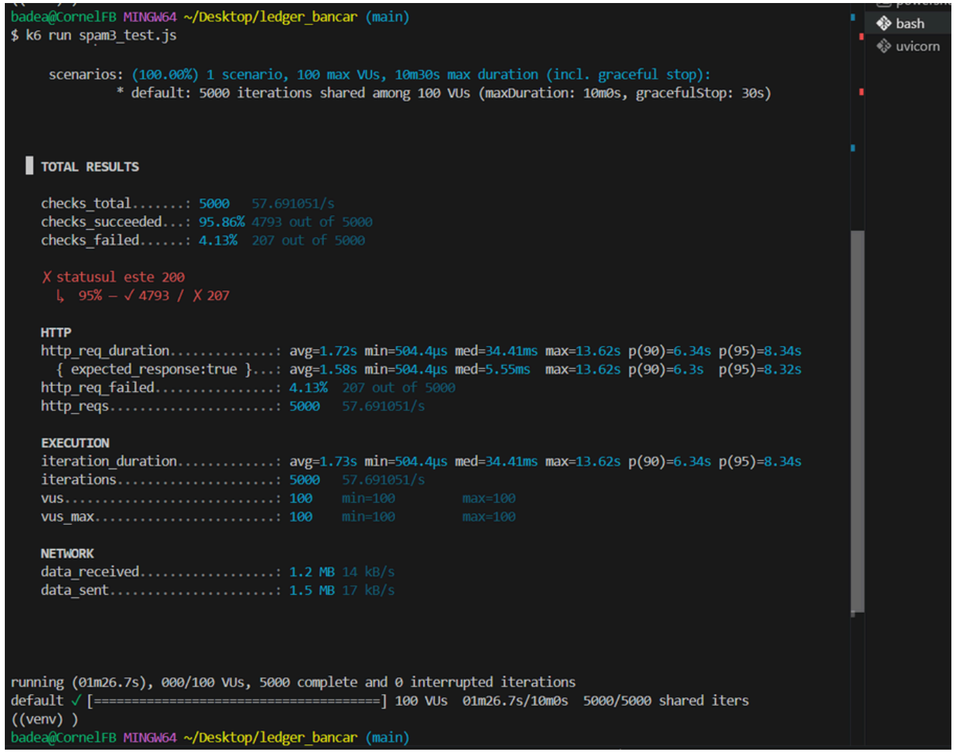
For this scenario, I pushed the system to its absolute limits with a sustained endurance test. I configured K6 to send 100,000 requests. As you can see in the terminal, the test ran for over 10 minutes until it hit the maximum duration timeout, successfully sending over 22,000 requests with a 96.7% server response rate. > However, the most important metric is the database audit. Despite the extreme concurrency and hardware bottlenecks, the database successfully processed exactly 21,850 transactions. As shown on the dashboard, the total system funds remained exactly at 300,000 cents. There were zero race conditions, zero lost updates, and the ledger remained perfectly balanced throughout the entire attack. This mathematically proves that the Pessimistic Locking architecture is impenetrable.

retty-print

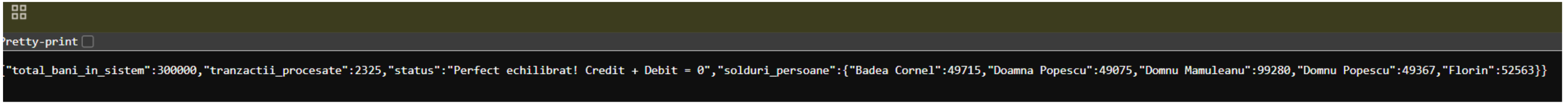
"total_bani_in_sistem":300000,"tranzactii_procesate":21850,"status":"Perfect echilibrat! Credit + Debit = 0","solduri_persoane":{"Badea Cornel":50315,"Doamna Popescu":48399,"Domnu Mamuleanu":101960,"Domnu Popescu":50284,"Florin":49042}}

TEST 3: THE COMBO TEST





For the final test, I designed a 'Combo Attack' that mimics real-world chaos. K6 was configured to send 5,000 requests, mixing randomized valid transactions with intentional duplicate spam keys across all accounts simultaneously.
Looking at the terminal, the server handled the load exceptionally well, with a 95.8% response rate. However, the true strength of the architecture is visible in the audit.
The database only executed 2,325 transactions. The system successfully identified and blocked the remaining 2,400+ requests because they were either duplicate spam keys caught by Redis, or invalid transfers caught by our Pessimistic Locking logic. Most importantly, the total system funds remained exactly at 300,000 cents. The ledger is perfectly balanced. This final test conclusively proves that the combination of Redis caching and PostgreSQL row-level locking creates a highly secure and impenetrable financial engine.



conclusions

- Architectural Resilience: The combination of FastAPI, PostgreSQL, and Redis proved highly effective in mitigating extreme concurrent loads.
- 100% Data Integrity: Across all stress tests (Spam, Chaos, and Combo), zero cents were lost or duplicated. The core rule of Credit + Debit = 0 was strictly maintained.
- Zero Race Conditions: Database-level Pessimistic Locking (FOR UPDATE) successfully prevented "phantom money" during simultaneous writes.
- Impenetrable Shield: Redis-backed Idempotency filtered out thousands of malicious or accidental duplicate requests before they could tax the main database.
- Final Verdict: The implemented architecture behaves precisely like a real-world banking engine, prioritizing absolute security and consistency over raw, unprotected speed.